Diretrizes Oficiais e Baseline do Projeto

O Algoritmo FCM Puro (*Baseline*) e o Controle de Bancada

Projeto Final de Engenharia de Programas

Prof. Dr. Diego Gervasio Frias Suarez

Semestre Letivo de 2026

Adendo Especial: Obrigatoriedade do Baseline (O Código "Puro")

Para que a aferição analítica através da Metodologia F.O.C.A. possua validade científica e rigor estatístico, o projeto prático deste Crédito 2 operará como um experimento físico estritamente controlado. É **terminantemente proibido** o desenvolvimento do algoritmo a partir do zero ou a adoção de estruturas arquiteturais livres.

Todas as doze equipes deverão utilizar, de forma obrigatória, o script `fcm_puro.py` (apresentado na Seção seguinte) fornecido pela coordenação da disciplina como seu mapa base inegociável (*baseline*).

Diretrizes de Intervenção no Código:

- **Ação Cirúrgica:** As modificações, supressões ou injeções de código devem se restringir **exclusivamente** às linhas e laços lógicos necessários para forçar o cumprimento da Restrição Arquitetural sorteada para a equipe.
- **Preservação de Escopo:** Fases do algoritmo que não são alvos diretos da missão da equipe devem permanecer estruturalmente idênticas ao código Puro original.
- **Isolamento de Variáveis:** Esta imposição visa impedir a contaminação do polinômio empírico $T(n, K)$ por ruídos de engenharia de software (como más práticas de alocação de memória ou laços redundantes não intencionais).

Nota do Tribunal Arquitetural: O descumprimento desta norma configurará "adulteração de bancada". A garantia de que todos partam do mesmo motor assegura que qualquer anomalia gráfica ou explosão na variância do tempo de execução seja o reflexo direto, isolado e irrefutável do gargalo de hardware provocado pela missão da equipe.

Código Oficial de Referência: `fcm_puro.py`

Abaixo consta o código-fonte basal que serve como padrão-ouro para o desenvolvimento das variantes arquiteturais.

```python
import random

def fcm_puro(X, K, m=2.0, max_iter=100, epsilon=0.01):
    """
    Implementacao FCM "Puro" (Baseline) sem bibliotecas avancadas
    ou funcoes escuras (utilizado por todas as equipes como base).
    X: Imagem (Lista de pixels, onde cada pixel e [R, G, B])
    K: Numero de clusters
    m: Fator de fuzzificacao (padrao 2.0)
    """
    N = len(X)
    canais = len(X[0]) # Tipicamente 3 (RGB)

    # FASE 1: INICIALIZACAO DA MATRIZ DE PERTINENCIA (U)
    U = []
    for i in range(N):
        linha = []
        soma_linha = 0.0

        for j in range(K):
            val = random.random()
            linha.append(val)
            soma_linha += val

        for j in range(K):
            linha[j] = linha[j] / soma_linha

        U.append(linha)

    # LACO PRINCIPAL DO FCM
    for iteracao in range(max_iter):

        # FASE 2: ATUALIZACAO DOS CENTROIDES
        C = []
        for j in range(K):
            numerador = [0.0] * canais
            denominador = 0.0

            for i in range(N):
                u_m = U[i][j] ** m
                denominador += u_m

                for c in range(canais):
                    numerador[c] += u_m * X[i][c]

            centroide_j = []
            for c in range(canais):
                centroide_j.append(numerador[c] / denominador)
            C.append(centroide_j)

        # FASE 3 E 4: ASSOCIACAO (DISTANCIA) E CONVERGENCIA
```

```
        U_nova = []
        max_diff = 0.0

        for i in range(N):
            linha_nova = [0.0] * K
            distancias = [0.0] * K

            for j in range(K):
                soma_quadrados = 0.0
                for c in range(canais):
                    diff = X[i][c] - C[j][c]
                    soma_quadrados += diff * diff

                distancias[j] = soma_quadrados ** 0.5

            for j in range(K):
                if distancias[j] == 0.0:
                    linha_nova[j] = 1.0
                    for k in range(K):
                        if k != j:
                            linha_nova[k] = 0.0
                    break

                soma_dist = 0.0
                for k in range(K):
                    razao = distancias[j] / distancias[k]
                    soma_dist += razao ** (2.0 / (m - 1.0))

                pertinencia = 1.0 / soma_dist
                linha_nova[j] = pertinencia

                diff_u = pertinencia - U[i][j]
                if diff_u < 0:
                    diff_u = -diff_u

                if diff_u > max_diff:
                    max_diff = diff_u

            U_nova.append(linha_nova)

        U = U_nova

        if max_diff < epsilon:
            break

    return C, U
```